

Project Report

Course: Imperative programming

Astrophysics and Astronomy: Analysis and Simulation

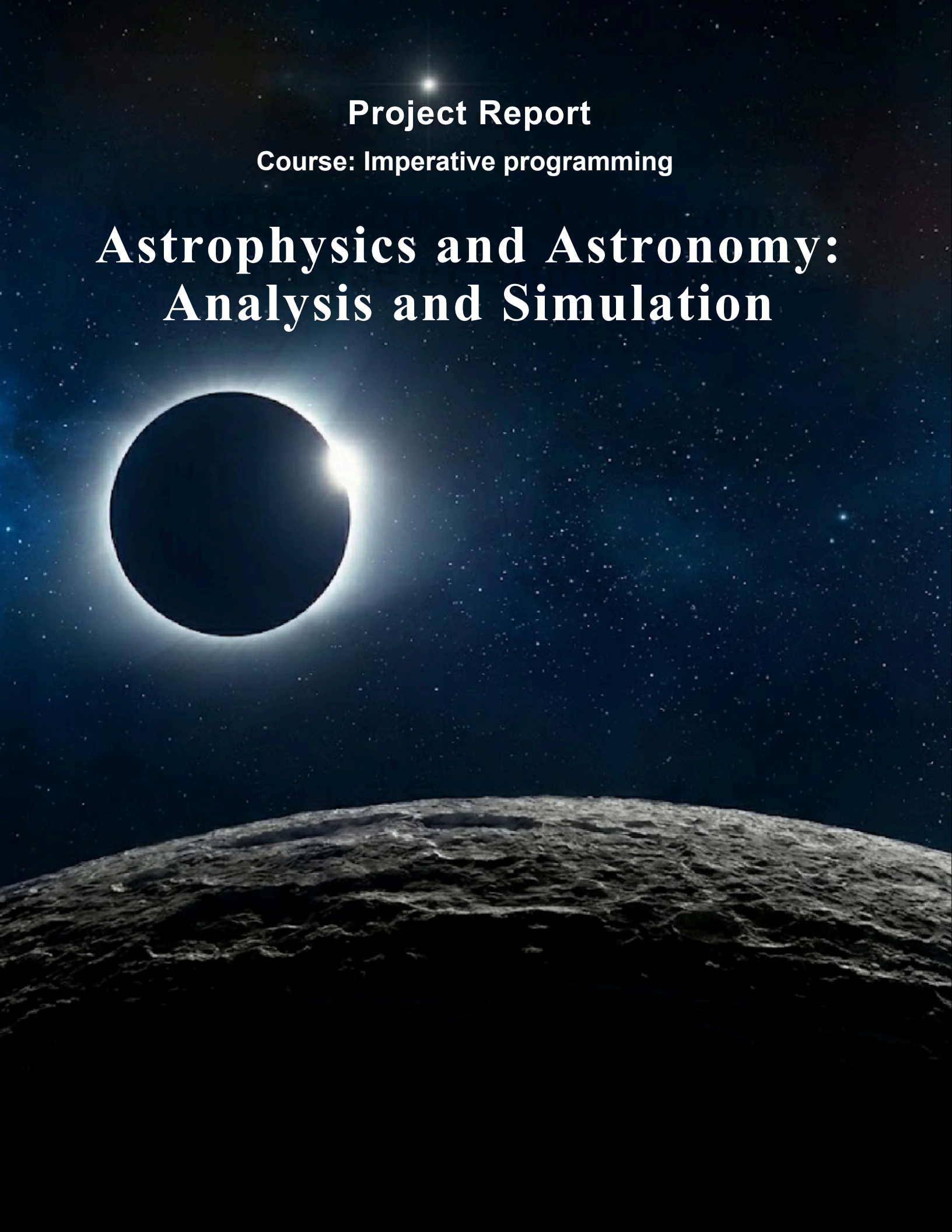


Table of Contents

1. Introduction.....	3
2. Project Overview	4
I) Analysis Section	4
II) Simulation Section	13
3. Technical Choices	16
4. Code Architecture and Laws Used	19
5. Conclusion	30
6. Appendices	31

Introduction

For a long time, humanity has sought to understand the universe and how it works. This has become possible through astronomy and astrophysics, which allow us to study the movements of celestial bodies, the physical laws that govern space, as well as the relationships between different characteristics of stars and planets.

These fields are very interesting and appealing because they allow us to uncover the mysteries of space.

Today, these sciences rely heavily on computer science to process data, analyze phenomena, and perform useful simulations.

In this project, carried out as part of the Imperative programming course, we used the Python language to analyze real astrophysical data and simulate the motion of celestial bodies under the influence of gravity.

The goal is to apply computational and modeling methods to better understand certain laws of the universe.

This report presents the stages of our work, our research, our technical and theoretical choices, as well as the results obtained and a detailed description of our code.

Project Overview

This project consists of two main parts: the analysis of real astrophysical data and a numerical simulation of the solar system.

I) Part analysis

The first part is devoted to verifying the validity of certain fundamental physical laws and relationships by analyzing real data. We studied the following relationships:

1) Kepler's Third Law:

Also known as the “Law of Periods,” this is one of the laws we studied in school many times, discovered by Johannes Kepler in 1618. It describes the proportionality between the square of the period (T) and the cube of the semi-major axis (a) of the celestial body's elliptical orbit. We verified this law using data from Jupiter's moons and the planets of the solar system, by applying a linear regression to the relationship $T^2 = f(a^3)$.

- Results obtained:

For the data on Jupiter's moons, we first calculated the theoretical constant that satisfies $T^2 = k \cdot a^3$, which is:

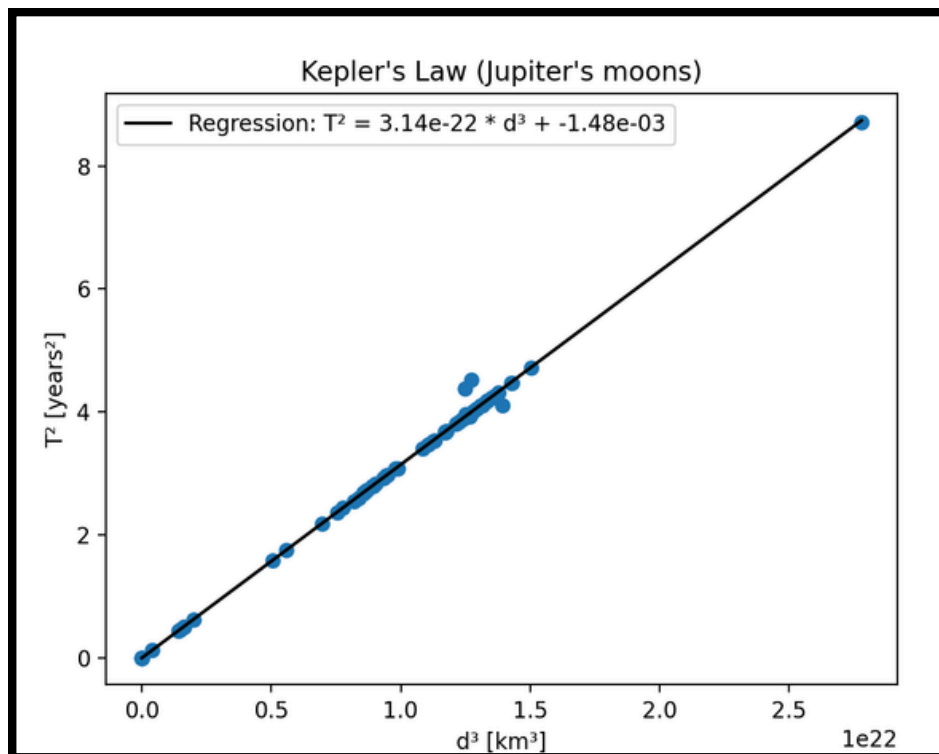
$$k = \frac{4\pi^2}{GM_j}$$

Where G is the gravitational constant ($G = 6.674 \times 10^{-11} \text{ m}^3 \cdot \text{kg}^{-1} \cdot \text{s}^{-2}$) and M_J is the mass of Jupiter ($M_J = 1.898 \times 10^{27} \text{ kg}$)

We calculated that $k = 3.1336168 \times 10^{-22}$.

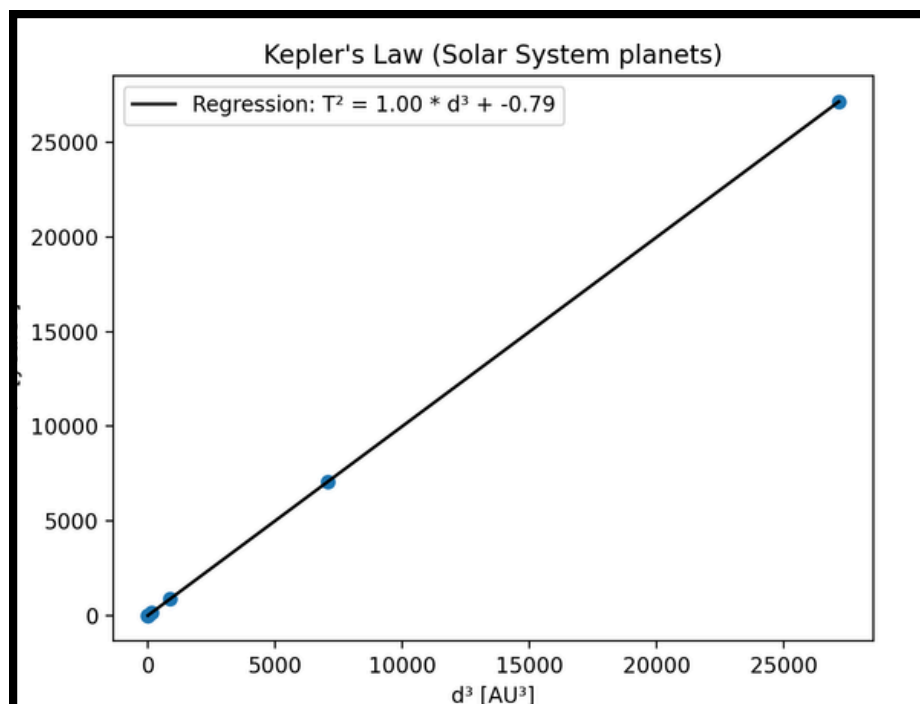
Next, we compared our regression results with the theoretical ones. The regression gave us a curve with the equation $T^2 = 3.14 \times 10^{-22} \cdot a^3 - 0.001$, which is very close to the theoretical value of the constant k.

Figure 1: Kepler's Law — Jupiter's Moons



We applied the same approach to the planets of the solar system, this time using (AU) for distances and years for periods, which simplifies the theoretical coefficient k to 1. The linear regression yielded a straight line with the equation $T^2 = 0.9998 * a^3 - 0.793$, which is consistent with theory and confirms the validity of Kepler's third law.

Figure 2: Kepler's Law — Planets of the Solar System



2) Relationship between temperature and distance:

This law represents the relationship between a planet's equilibrium temperature and its distance from its star. Indeed, the closer the planet is to the star, the more more radiant energy, and thus its temperature becomes higher. From a theoretical standpoint, the formula for temperature of a planet, assuming that its only source of light is its star, is:

$$T = \left(\frac{L (1 - a)}{16\sigma\pi d^2} \right)^{\frac{1}{4}}$$

Where L is the star's luminosity

σ : Stefan-Boltzmann constant = 5.67×10^{-8} W·m⁻²·K⁻⁴, relates the temperature of a blackbody (whose sole source of light is its star) to the energy it radiates.

a : Albedo of the planet: fraction of light reflected by the planet

d: Distance between the planet and its star

This relationship indicates that a planet's equilibrium temperature is proportional to $d^{-0.5} \rightarrow \frac{1}{\sqrt{d}}$

neglecting the effects of luminosity (L) and albedo (a).

- Method

To verify this relationship, we used data from NASA's official archive containing the properties of confirmed exoplanets. To avoid planets with unusual behavior, we applied two filters to the data:

- Keep only exoplanets located within 4 AU of their star, to remove very distant gas giants.
- Keep only exoplanets with masses greater than 5 Earth masses, to focus on sufficiently massive objects and avoid rocky ones whose temperature depends heavily on (atmosphere, albedo).

To perform a linear regression on such a power-law relationship, we transformed the data into a logarithmic scale

$$T = C.d^{-0.5} \Rightarrow \log(T) = \log(C.d^{-0.5})$$

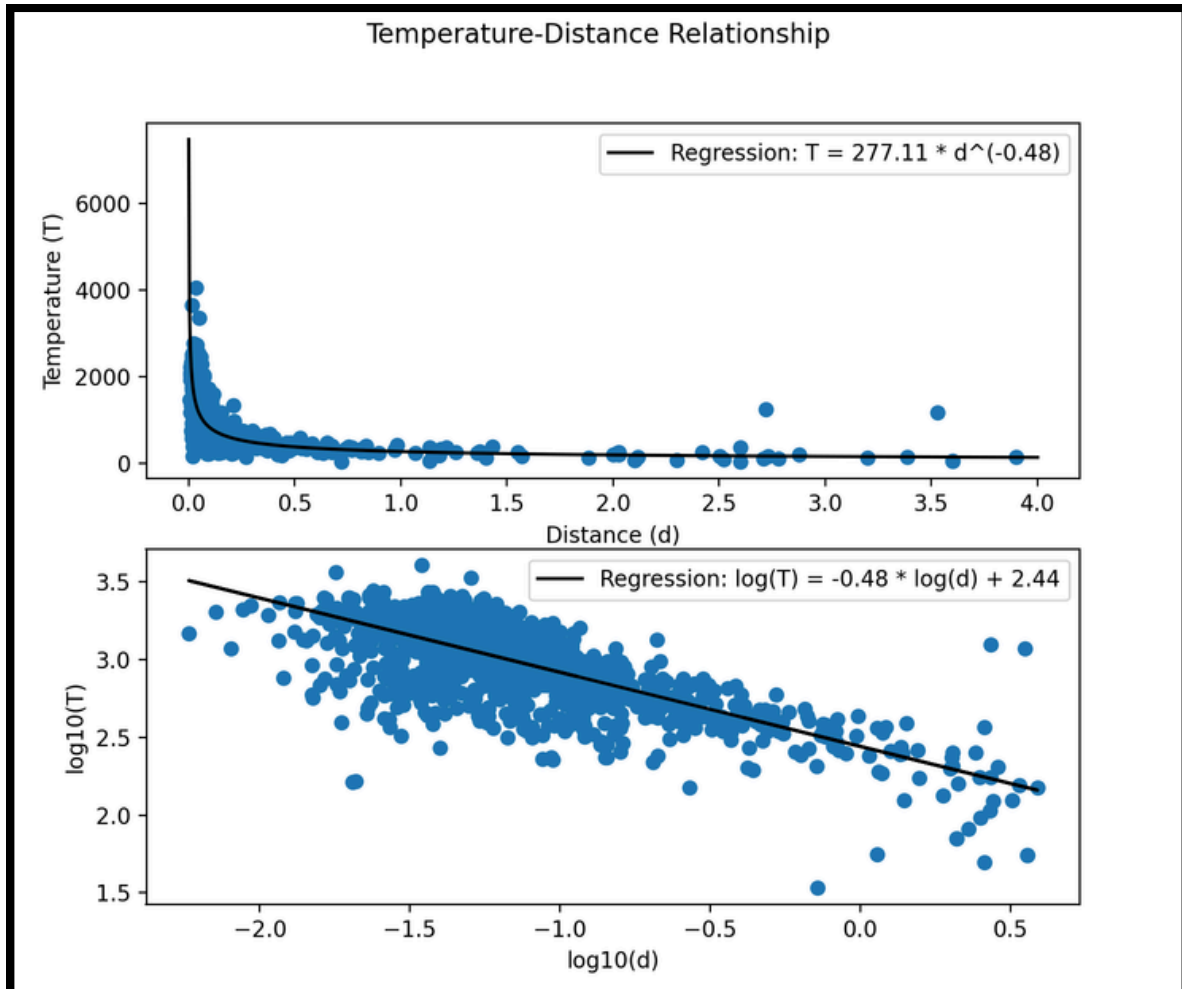
$$\Rightarrow \log(T) = \log(C) - 0.5 \times \log(d)$$

This transformation allows us to obtain a linear relationship to which we can apply the regression of the form $y = ax + b$.

- Results obtained:

The regression yields the relationship $T = 277.11 \times d^{(-0.48)}$, which is close to the theoretical relationship $T \propto d^{(-0.5)}$, with an exponent of -0.48 very close to -0.5 , confirming the validity of the theoretical relationship between temperature and distance.

Figure 3: Results of the analysis of temperature and distance data for exoplanets



3) Relationship between the radius and mass of stars:

The mass-radius relationship is the link that characteristically relates characteristically links the mass and radius of an object based on its composition and structure. In general, main-sequence stars (which are stable) follow a relationship of the following type:

$$R = M^{\xi}$$

where M and R are the mass and radius of the star in question, expressed in solar masses and solar radii, respectively.

However, two regimes (two values of ξ) can be distinguished depending on the mass range to which the star belongs; the "break" occurs around the mass of the Sun.

- for stars less massive than the Sun ($M < 1$):

$$R = M^{0.8}$$

- for stars more massive than the Sun ($M > 1$):

$$R = M^{0.57}$$

- Method

First, we applied a logarithmic transformation to the data, since this is a power-law relationship. However, we noticed that some stars are very far from the

main scatter plot in the graph showing radius versus mass. These are likely unstable stars and special cases. To remove these stars, we used a function that detects outliers on a logarithmic scale.

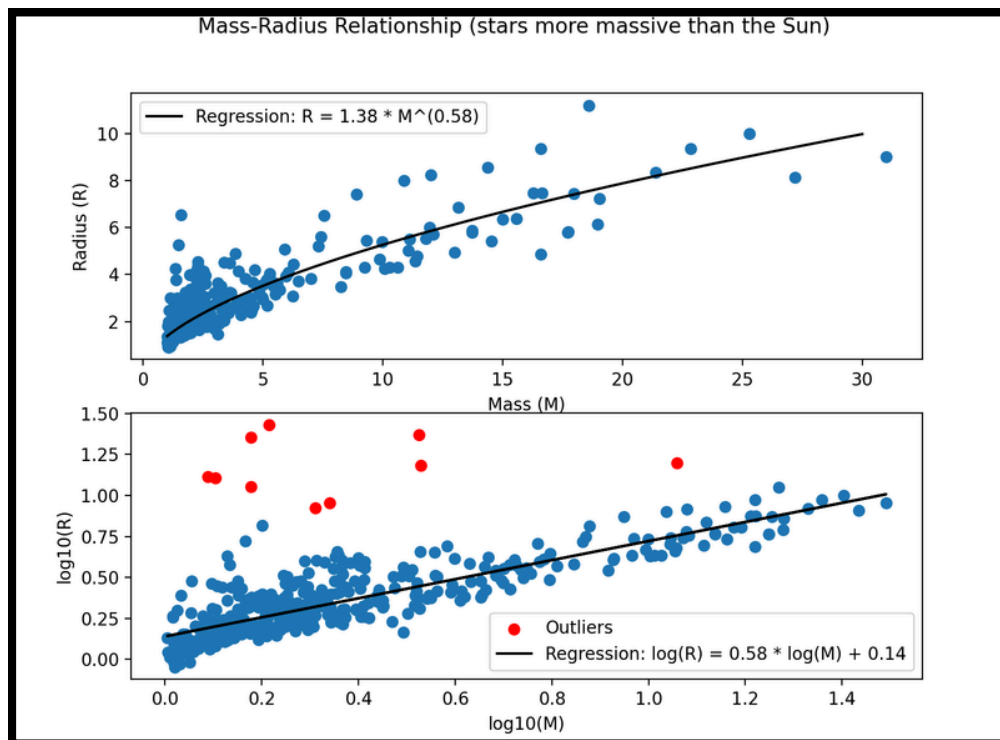
- Results

1. Stars more massive than the Sun

Regression result: $R = 1.38M^{0.58}$

This result is very close to the theoretical relationship: $R = M^{0.57}$

Figure 4: Results of the analysis of the mass-radius relationship for stars more massive than the Sun

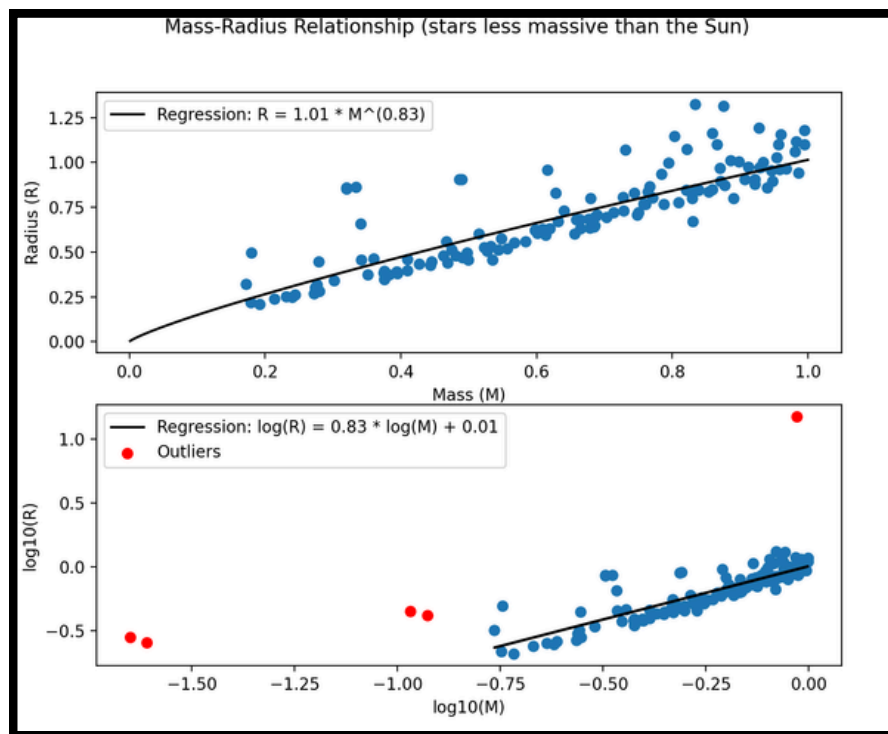


2.Stars less massive than the Sun

Regression result: $R = 1.01M^{0.83}$

This result is very close to the theoretical relationship: $R = M^{0.8}$

Figure 5: Results of the analysis of the mass-radius relationship for stars less massive than the Sun



The analysis of the actual data shows a trend very close to the theoretical law, whether for stars more massive than the Sun or those less massive.

II) Simulation Section

1) Overview

For the second part of this project, we created a dynamic simulation of the trajectories of the planets in the solar system in the form of an animation. The goal of this simulation is to provide a better visual understanding of the evolution of positions of the planets over time, the effect of the Sun's gravitational pull on these planets, and the gravitational pull of other celestial bodies. It also allows us also to verify the validity of certain laws, such as Kepler's laws, to observe the nature of the planets' trajectories, and the differences between their speeds and orbital periods. Our simulation is not limited to the 8 planets of the solar system; we have also added other planets that are randomly generated. We have also included a probability that asteroids will appear. This allows us to observe the stability of the solar system and how it reacts to external perturbations.

2) How the simulation works

The simulation involves calculating the position of each object at each time step. To make these calculations

as accurate as possible, we used real data from the NASA website for the initial conditions, and the Verlet method to calculate the positions of the bodies over time. Details of this method will be presented in the section <Code Architecture and Laws Used>.

3) Collisions and Mergers

The presence of new external bodies can disrupt the trajectories and velocities of objects due to gravitational attraction, there by increasing the risk of collisions. These collisions are handled in the simulation using the principle of conservation of momentum by merging objects in contact.

4) Display management in the simulation

In the simulation, the time is displayed directly in the top-left corner and is continuously updated. This allows you to track the system's evolution by knowing the current simulation time.

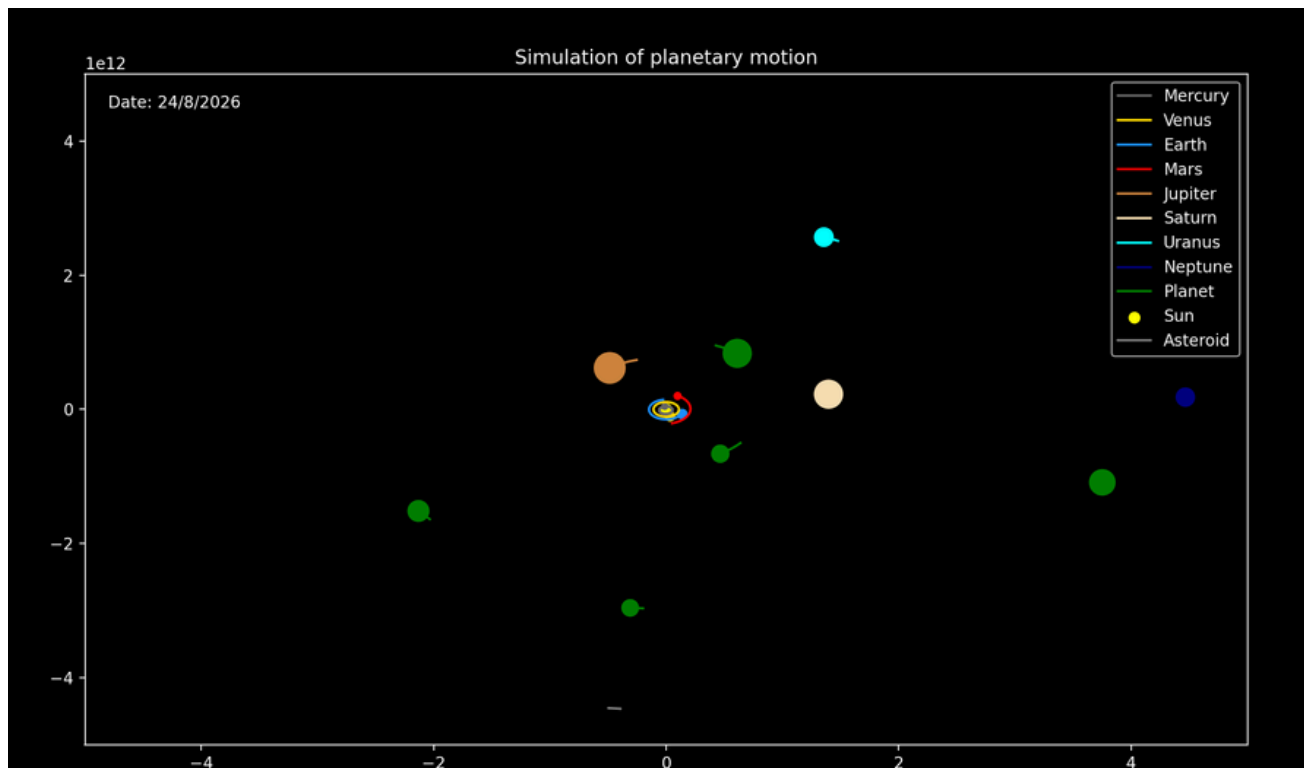
Objects are depicted using their actual radius, which allows for the differences in size between celestial bodies to be respected and results in a visualization more realistic.

5) Observations

The simulation allowed us to make several observations. First, we noticed that the planets and asteroids closer to the Sun orbit faster than the others, their orbital periods are therefore shorter, which is consistent with Kepler's third law.

Second, the planets' trajectories are not perfectly circular but follow an elliptical path, which is consistent with Kepler's first law. As for collisions, they remain rare under normal conditions and could only be observed by increasing the number of random planets and the probability of asteroids appearing.

Figure 6: Screenshot illustrating the solar system simulation



Technical Choices

In this section, we will outline the technical choices made during the project's development, as well as the reasons behind these choices.

1) Choice of libraries

- **Matplotlib:** chosen for its ability to visualize scientific data and generate dynamic simulations. It allowed us to create the graphics and the animation for the planetary system simulation.
- **Numpy:** allows for the simple, efficiently, and quickly perform vector calculations, as well as mathematical operations such as logarithms, which are essential for and regressions.
- **CSV:** this module allows for the manipulation of files in CSV format. The choice to use this format was driven by its ease of reading and the ease with which it allows the necessary data to be extracted.
- **Random:** chosen to introduce a random component into the simulation, allowing for the generation of asteroids and planets with varying characteristics and making the system more realistic by adding perturbations.

2) Choice of data sources

The data used in this project comes from several reliable scientific websites specializing in astronomy.

- For Jupiter's moons, the data comes from a scientific GitHub repository containing precise orbital measurements.
- For the planets of the solar system, we used a database dedicated to planetary orbital parameters.
- For exoplanets (temperature, distance, and mass), the data comes from the Exoplanet Archive, a leading resource in the study of planetary systems.
- For stellar properties (mass and radius), the data comes from the VizieR database at Strasbourg Observatory.
- The initial positions and velocities of the planets in the simulation are based on the ephemerides from NASA's JPL Horizons system.

3) Logarithmic scaling

To study certain power-law relationships, we transformed the data to a logarithmic scale. This method involves taking the logarithm of the quantities in order to linearize the relationship. This allows us to transform an equation of $y = bx^a$ into a linear relationship of the form

$\log(y) = a \cdot \log(x) + \log(b)$, which allows us to apply linear regression.

4) Choice of physical laws and methods

In this project, we used several physical laws and methods to model the motion of bodies in a way that closely approximates reality. The most essential is Newton's law of universal gravitation, as well as the Verlet method, which allows us to calculate the positions of objects with precision. Additionally, we used the principle of conservation of momentum to correctly simulate the collision of bodies.

5) Reducing the Number of Saved Positions

When performing calculations with very large NumPy arrays, the computer's memory is heavily utilized, which slows down the animation and the computer's overall performance. To avoid this problem, we have limited the size of NumPy arrays to 500 elements, which allows us to simulate multiple bodies over long periods of time while maintaining good performance.

Code architecture and laws used

1) General Structure

The code is organized into two main parts: a **data analysis section** (lines 1–259) and a **simulation section** (lines 260–600).

2) Key functions

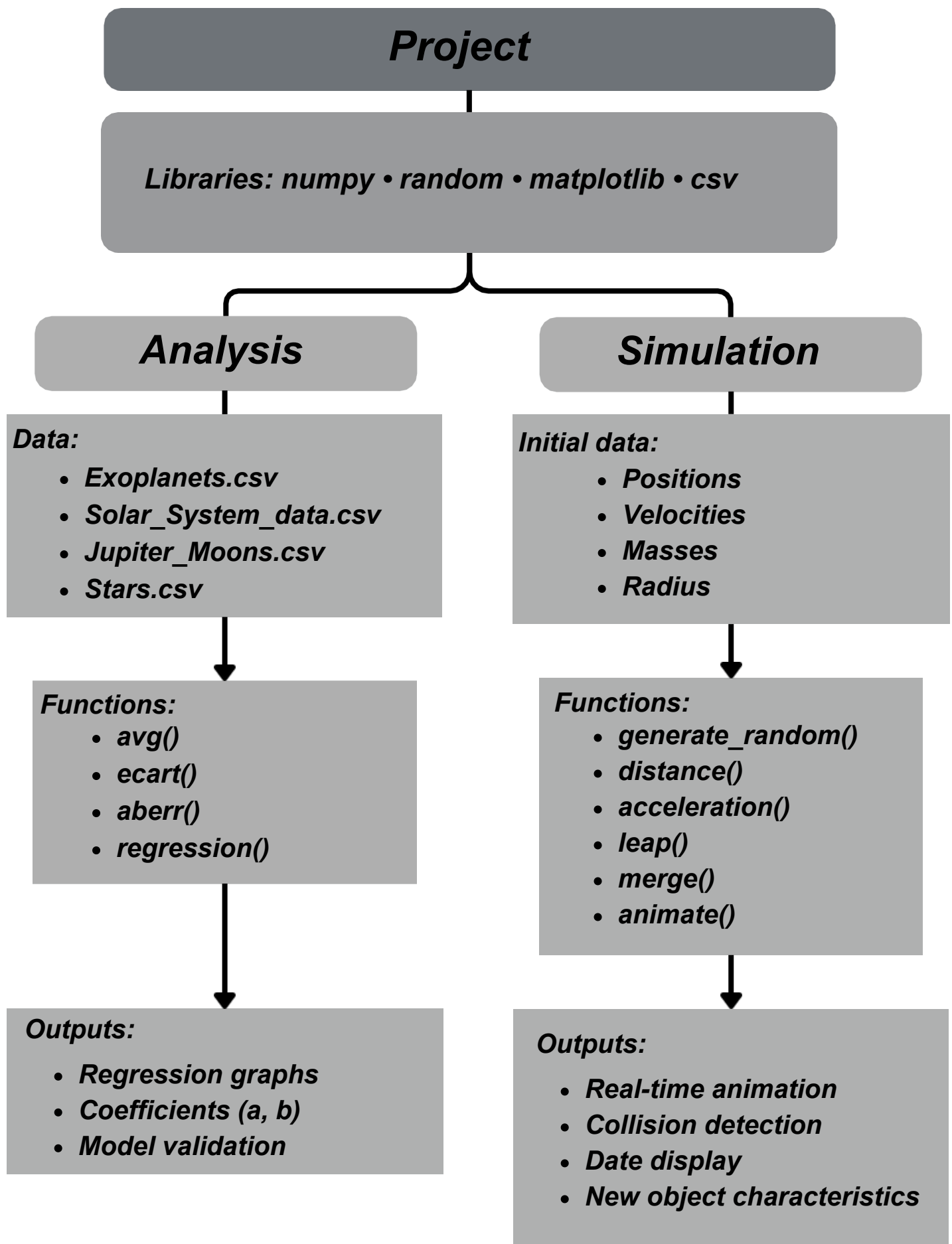
Function	Description
avg(x)	Computes the mean of a dataset
ecart(x)	Computes the standard deviation
aberr(x, y, a, b)	Detects outliers based on defined criteria
regression(x, y)	Performs linear regression (least squares method)
distance(i, j)	Computes the distance between two bodies (Pythagorean theorem)

Function	Description
acceleration(j)	Computes the gravitational acceleration of object j
generate_random(choice)	Generates a random planet or asteroid with realistic parameters
merge(survivor, vanishing)	Merges two colliding objects
animate(i)	Main animation loop function
leap(i)	Checks if a year is a leap year

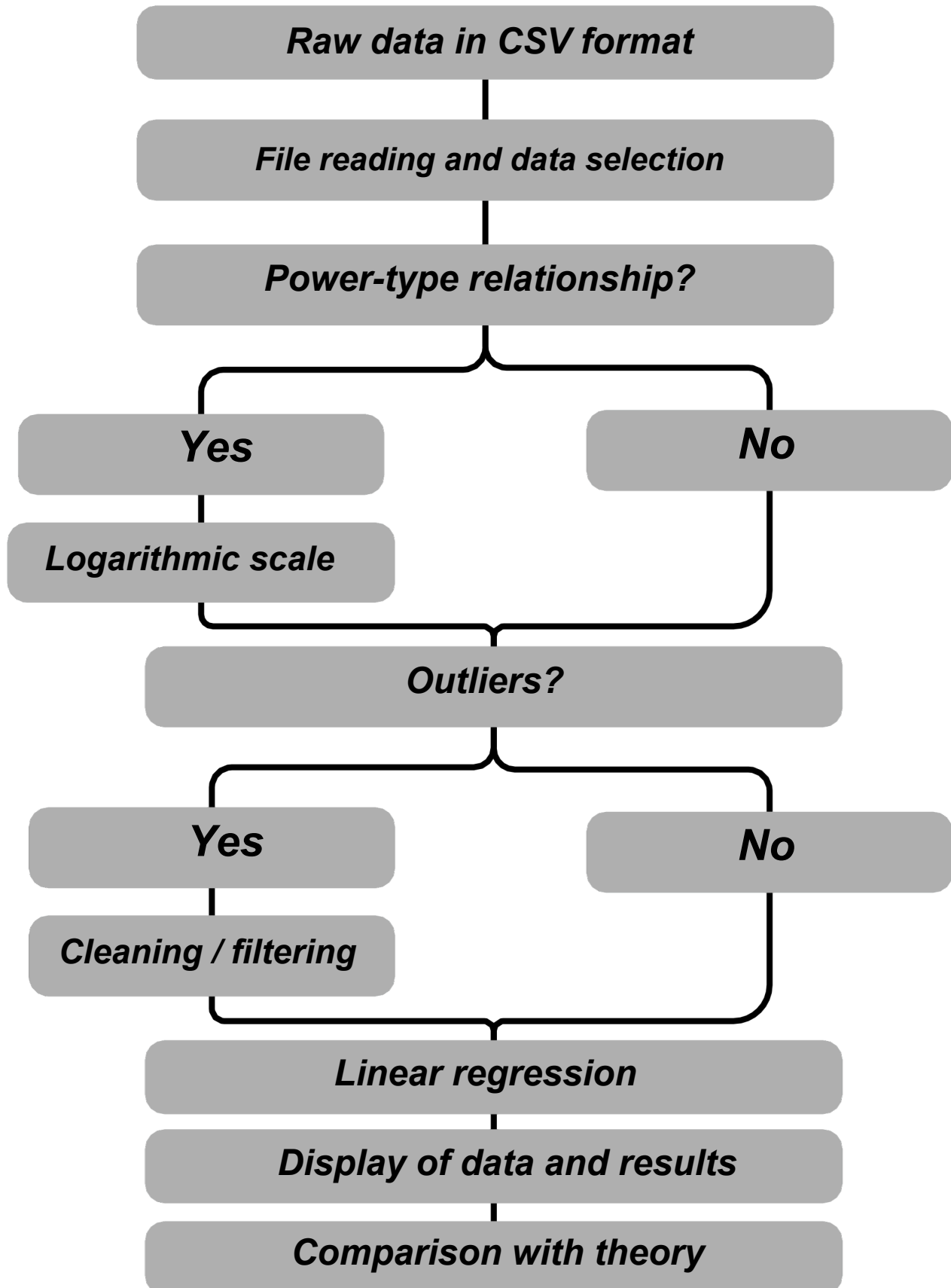
3) Architectural diagrams

3.1) Overview of the sections

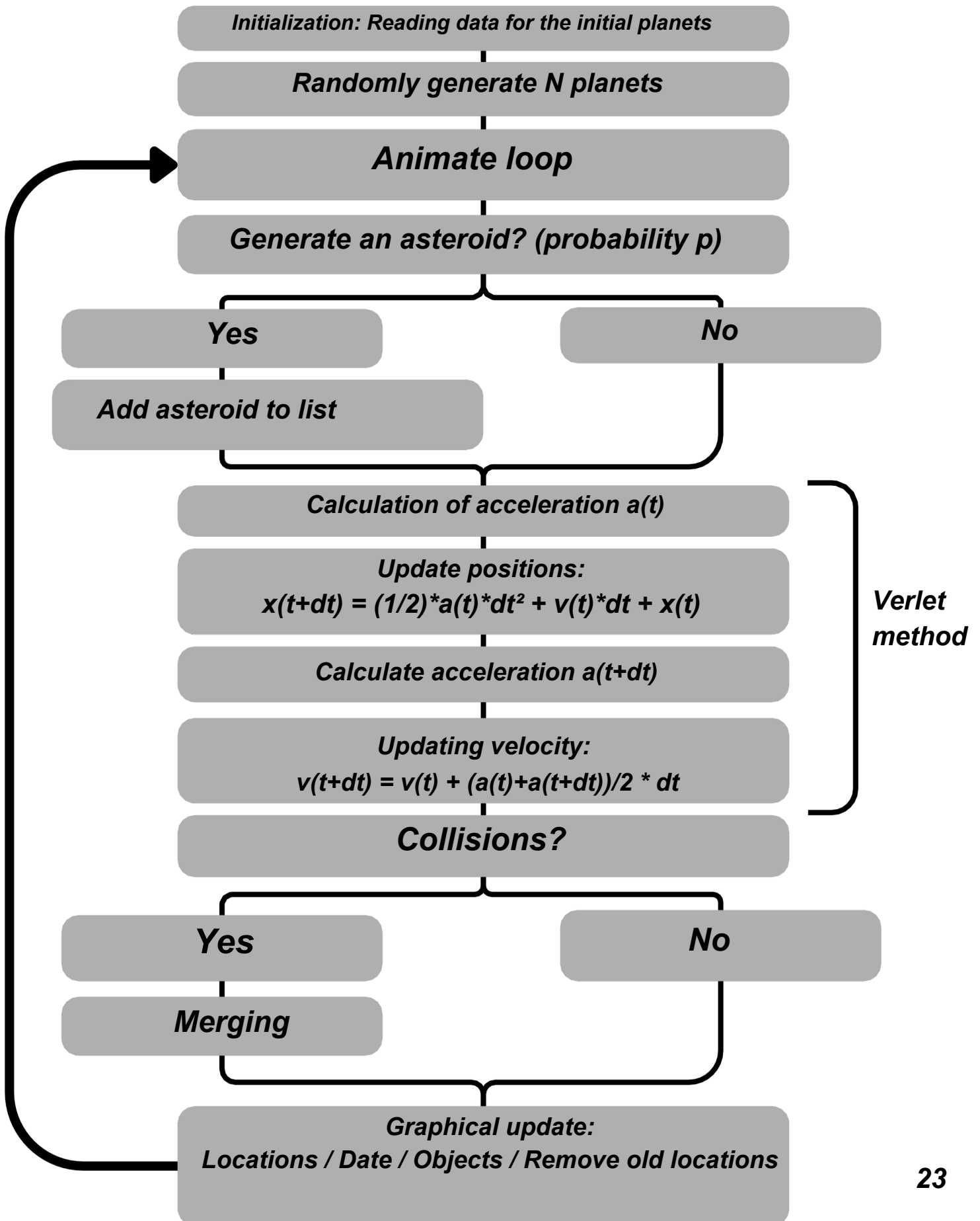
The diagram below illustrates the general organization of the program into two parts:



3.2) Data analysis process diagram



3.3) Simulation loop algorithm



4) Data structure

Each celestial object is represented by a dictionary of the following form:

```
{  
    "name": string,          # Object name  
    "r": float,              # Radius (m)  
    "m": float,              # Mass (kg)  
    "p": np.array([[X, Y]]), # Position history  
    "v": np.array([Vx, Vy]), # Current velocity  
    "color": string          # Color for display  
}
```

This structure allows direct and efficient access to the properties of each object. The dictionaries are then stored in a list called ***planets***.

5) Laws used in the simulation

After defining the software architecture and data structure, we will detail the physical laws that govern the interactions between celestial objects within the simulation loop.

5.1) Newton's law of universal gravitation

Two bodies i and j with masses m_1 and m_2 separated by a distance r exert an attractive force on each other, given by the formula:

$$\vec{F} = -G \frac{m_i \cdot m_j}{r^2} \vec{u}$$

Where $\vec{u}$: Unit vector pointing from i to j

$\vec{F}$: Gravitational force exerted by body i on j

G: Gravitational constant

m_j, m_i : masses of the bodies

r: distance between the two bodies

The unit vector is expressed in terms of the position vector:

$$\vec{u} = -\frac{\vec{r}}{r}$$

Where $\vec{r} = \vec{r}_i - \vec{r}_j$: Position vector pointing from j to i

$r = \|\vec{r}\|$: Norm of the position vector

Substituting into the law of gravitation:

$$\vec{F} = G \frac{m_i \cdot m_j}{r^3} \vec{r}$$

5.2) Newton's Second Law (Fundamental Principle of Dynamics)

This law states that the sum of the forces acting on an object is equal to the product of its mass and its acceleration. Mathematically, the formula is:

$$\sum \vec{F} = m_j \vec{a}$$

This formula helps us calculate the acceleration of objects in our simulation by substituting the force with its formula:

$$\vec{a}_j = \frac{1}{m_j} \sum_{i=1}^n G \frac{m_i m_j}{r^3} \vec{r} = \sum_{i=1}^n G \frac{m_i}{r^3} \vec{r} = \sum_{i=1}^n G \frac{m_i}{r^3} (\vec{r}_i - \vec{r}_j)$$

Where $\vec{a}_j$: the acceleration of body j

n: the number of bodies exerting a force on j

Implementation in the code:

```
def acceleration(j):
    a = np.array([0.0, 0.0])
    for i in range(len(planets)):
        if i != j:
            ai = G * planets[i]["m"] / distance(i, j) ** 3 * (planets[i]["p"][-1] - planets[j]["p"][-1])
            a += ai
    return a
```

5.3) Verlet integration method

The Verlet method is a numerical integration algorithm that solves the equations of motion while better preserving the total energy of the system than the simple Euler method.

Formulas:

Step 1: Calculate the new position

$$\vec{r}_j(t + \Delta t) = \frac{1}{2} \vec{a}_j(t) \times \Delta t^2 + \vec{v}_j(t) \times \Delta t + \vec{r}_j(t)$$

Step 2: Calculate the new acceleration

$$\vec{a}_j(t + \Delta t) = \sum G \frac{m_i}{r^3} [\vec{r}_i(t + \Delta t) - \vec{r}_j(t + \Delta t)]$$

Step 3: Update the velocity to use in the next position

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{1}{2} [\vec{a}(t) + \vec{a}(t + \Delta t)] \times \Delta t$$

Implementation in code:

```
a = np.array([acceleration(j) for j in range(len(planets))])

for n in range(len(planets)):
    pos = a[n] / 2 * dt ** 2 + planets[n]["v"] * dt + planets[n]["p"][-1]
    planets[n]["p"] = np.append(planets[n]["p"], [pos], axis=0)

a_new = np.array([acceleration(j) for j in range(len(planets))])

for k in range(len(planets)):
    velocity = planets[k]["v"] + ((a[k] + a_new[k]) * dt) / 2
    planets[k]["v"] = velocity
```

5.4) Conservation of momentum

In an isolated system (without external forces), total momentum is conserved during a collision. This gives us a key relationship for handling contacts between bodies correctly:

$$\vec{p}_{before} = \vec{p}_{after}$$

where $\vec{p}_{before}$ and $\vec{p}_{after}$ are the total momenta of the system before and after the collision.

Now, the momentum of a body is defined by: $\vec{p} = m\vec{v}$

This gives for a collision between two bodies i and j:

$$m_j \vec{v}_j + m_i \vec{v}_i = (m_i + m_j) \vec{v}_{new}$$

$$\Rightarrow \vec{v}_{new} = \frac{m_j \vec{v}_j + m_i \vec{v}_i}{(m_i + m_j)}$$

Implementation in the code:

```
def merge(survivor, vanishing):

    m1 = float(planets[survivor]["m"])
    m2 = float(planets[vanishing]["m"])

    v1 = np.array(planets[survivor]["v"])
    v2 = np.array(planets[vanishing]["v"])

    r1 = planets[survivor]["r"]
    r2 = planets[vanishing]["r"]

    m_new = m1 + m2
    v_new = (m1 * v1 + m2 * v2) / m_new
```

5.5) Conservation of volume (approximation)

Assumption: During the merger, we assume that the total volume is conserved.

Formula: $V_t = V_1 + V_2$

The volume of a sphere is given by: $V = \frac{4}{3}\pi r^3$

Assuming that the planets are nearly spherical, this relationship allows us to calculate the new radius of the object resulting from the merger of two bodies.

$$\frac{4}{3}\pi r_f^3 = \frac{4}{3}\pi r_1^3 + \frac{4}{3}\pi r_2^3$$

$$\Rightarrow r_f^3 = r_1^3 + r_2^3$$

$$\Rightarrow r_f = \sqrt[3]{r_1^3 + r_2^3}$$

Implementation in the code:

```
planets[survivor]["r"] = (r1 ** 3 + r2 ** 3) ** (1 / 3)
```

Conclusion

Thanks to this project, we had the opportunity to explore the field of astrophysics and space using the Python language.

We were able to study laws related to this field, such as Kepler's third law, the relationship between the temperature and distance of exoplanets from their star, as well as the relationship between the mass and radius of stars. By using real data from reliable sources, we demonstrated the validity of these laws by performing linear regressions, which allowed us to reproduce the expected theoretical relationships.

At the same time, we developed a simulation of the solar system, based on the Verlet algorithm and real initial data from NASA, adding a probabilistic and random component, which gave us insight into how the solar system functions. Finally, this work helped us deepen our knowledge of astrophysics and also develop skills in data processing and analysis, numerical modeling, and Python programming. It also taught us to take a critical look at our results by identifying the limitations and approximations of our models, which is an essential skill in scientific research.

Appendices

Appendix 1: Data Sources Used

- **Kepler's laws**

Data on Jupiter's moons:

<https://gist.github.com/adrn/8151304#file-data-csv-L3>

Data on the planets of the solar system:

https://dataherb.github.io/flora/planets_in_solar_system/

- **Relationship between temperature and distance**

<https://exoplanetarchive.ipac.caltech.edu/cgi-bin/TblView/nph-tblView?app=ExoTbls&config=PS>

Official NASA/Caltech archive containing various physical properties of confirmed exoplanets

- **Relationship between mass and radius (stars)**

<https://vizier.cds.unistra.fr>

Astronomical catalog of the Strasbourg Observatory (CDS) listing the masses and radii of several stars.

- **Simulation**

<https://ssd.jpl.nasa.gov/horizons/app.html#/>

Jet Propulsion Laboratory (NASA) Horizons system providing the precise positions and velocities of bodies in the solar system in given moment.

Appendix 2: Sources of the new theoretical relationships used

- **Relationship between temperature and distance**

https://fr.wikipedia.org/wiki/Temp%C3%A9rature_d%27%C3%A9quilibre_%C3%A0_la_surface_d_un_planet

- **Relationship between mass and radius (stars)**

https://fr.wikipedia.org/wiki/Relation_masse-rayon

- **Verlet method**

<https://femto-physique.fr/analyse-numerique/methode-de-verlet.php>